

README

Darcy Liu (d273liu)

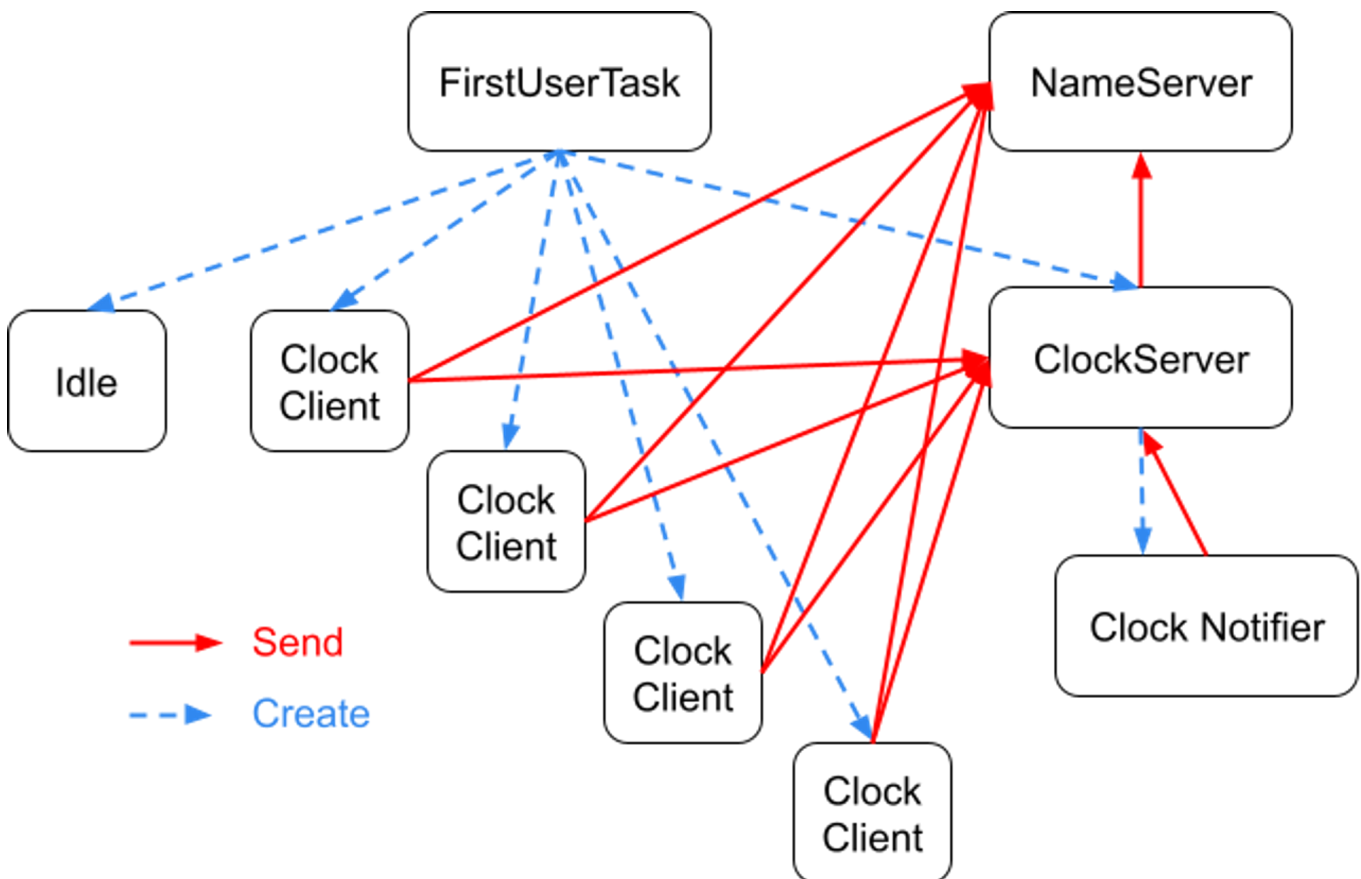
Repo name: cs452-trains

Commit Sha:

3065ce77b44e50eb17e4b7752cf8bfd51745601c

K3 Description:

Right now, the kernel consists of a name server that handles all the identity mapping, in addition to a clock server to keep track of the time. For the k3 assignment I have incorporated the structure as is:



I have 4 tasks that would send to the clock server and upon their “fire time” meaning they wake up, they would send a message to the terminal stating they have woken up and then immediately go to sleep.

Clock server output

The idle percentage stayed around 50% when the tasks are running 65% after the clock tasks are finished running. After the tasks have ended the percentage level gradually risen to 80%. One major contributing factor may be the clock tasks are printing to UART.

The way I measure time in my operation system is I took the difference between the clock time at the interrupt and the clock time before the context switch that ran the task. Thus I also need to keep context switch into account as well.

My command passing may also be sub-optimal as I used strings to denote the different types of commands. Thus string concatenation may take up some time

NameServer

This code represents a simple name server implemented in C for managing named processes. It allows processes to register, deregister, and query for other processes using human-readable names.

Code Structure

The code is divided into several sections:

- `nameserver` : The main function serving as the name server. It handles registration, deregistration, and query requests from processes. The name server maintains an array of process names and their associated PIDs.
- Utility Functions:
 - `RegisterAs` : A function for a process to register itself with the name server using a human-readable name.
 - `Deregister` : A function for a process to deregister itself from the name server.
 - `WhoIs` : A function for a process to query the PID of a process with a given name from the name server.
- Debugging and Terminal Control: The code includes terminal control sequences to improve the user interface. Debugging features are also integrated for monitoring and managing registered processes.

Usage

1. To register a process with a name, use the `RegisterAs` function. Example:
`RegisterAs("my_process_name");`
2. To deregister a process, use the `Deregister` function. Example: `Deregister();`
3. To query the PID of a process by its name, use the `WhoIs` function. Example:
`WhoIs("process_name_to_query");`
4. The `nameserver` function handles all the registration, deregistration, and query operations.

Rock paper Scissors server

1. Starting the Game Server:

- To begin, you need to start the game server. You can do this by running the command: `k2rps start`. This command initializes the RPS game environment.

2. Creating RPS Players:

- You can create RPS players by using the `k2rps create` command. The command follows the format: `k2rps create N type`, where:
 - `N` is the number of players you want to create (e.g., 1, 2, 3).
 - `type` specifies the type of player (options: "rock," "paper," "scissors," or "random").

Example: To create three players (one for each type - rock, paper, and scissors), you can run the following commands:

luaCopy code

```
k2rps create 1 rock k2rps create 2 paper k2rps create 3 scissors
```

3. Signup for the Game:

- Before playing, players need to sign up to join the RPS game. You can sign up by using the command: `k2rps signup`.

4. Playing the Game:

- Players can play the RPS game by using the `k2rps play` command, followed by their choice (rock, paper, or scissors).
- Example: To play "rock," you can use the command: `k2rps play rock`.

5. Quitting the Game:

- If you want to quit the game, you can use the `k2rps quit` command.

6. Managing the Game:

- To manage the game server, you can use the `k2rps` command with options like "start" (to initialize the game server) and "shutdown" (to shut down the game server).

Example:

- Starting the game server: `k2rps start`
- Shutting down the game server: `k2rps shutdown`

7. Additional Information:

- The code also includes functions for handling messages and tasks in the operating system, which are relevant for the game's implementation. These functions may be used internally by the game.

8. Command Syntax:

- Ensure that you follow the correct command syntax as per the examples provided to create players, play the game, or manage the game server.

Please note that this is a basic guide to get you started with using the code for the Rock, Paper, Scissors game. The code appears to be part of a larger system, so make sure you understand how it integrates with your operating system or program. Detailed documentation and integration instructions may be required depending on your specific use case.

Clock Server and Related Functions

This code contains a clock server and related functions to manage time and delays in a real-time system.

`clockNotifier()`

- **Description:** This function registers as "clockNotifier" and awaits events with an event type of 99. When such an event occurs, it sends a message to the clock server, requesting the current time.

`clock_server()`

- **Description:** The clock server handles time-related operations. It manages time ticks and processes incoming requests. The server can handle operations like querying the current time, delaying a task for a specified number of ticks, and delaying a task until a specific tick count is reached.
- **Functions:**
 - `Time(int tid)` : Queries the current time from the clock server.
 - `Delay(int tid, int ticks)` : Requests a delay for a specified number of ticks.
 - `DelayUntil(int tid, int ticks)` : Requests a delay until a specific tick count is reached.
- **Data Structures:**
 - `waketicks[NUMPROCS]` : An array that stores wake-up ticks for tasks. It's used to schedule and wake up tasks after specified delays.
- **Operation:**

- The clock server listens for incoming messages from tasks.
- It can process requests for time information or delay-related operations.
- The server maintains an array of scheduled wake-up times for tasks, allowing it to manage and trigger task delays efficiently.

`Time(int tid)`

- **Description:** A function to query the current time. It sends a request to the clock server for the current time.
- **Parameters:**
 - `tid`: The Process ID (PID) of the clock server.
- **Returns:** The current time retrieved from the clock server. Returns -1 on error.

`Delay(int tid, int ticks)`

- **Description:** A function to request a delay of a specified number of ticks. It sends a request to the clock server to delay the caller task.
- **Parameters:**
 - `tid`: The PID of the clock server.
 - `ticks`: The number of ticks to delay the caller task.
- **Returns:** The remaining ticks after the delay. Returns -1 on error.

`DelayUntil(int tid, int ticks)`

- **Description:** A function to request a delay until a specific tick count is reached. It sends a request to the clock server to delay the caller task until the specified tick count.
- **Parameters:**
 - `tid`: The PID of the clock server.
 - `ticks`: The target tick count to delay until.
- **Returns:** The remaining ticks after the delay. Returns -1 on error.

Helper Functions

`parse_char_arr(char *arr, char **num, int num_size)`

- **Description:** A helper function to parse a character array into an array of numeric strings. It is used in the clock server to parse incoming commands.

`i2a(int n, char *s)`

- **Description:** A helper function to convert an integer to a string. It is used to construct delay messages in the clock server.

Usage

To use the clock server and related functions, you can register tasks as "clockNotifier" to request the current time and use `Time`, `Delay`, and `DelayUntil` functions to manage delays and time-based operations in your real-time system.

Make sure to include the necessary header files such as "clockserver.h," "syscall.h," "processes.h," and others to use these functions effectively.

System Timer Control Documentation

This documentation provides an overview of the System Timer Control library for the Raspberry Pi, which is essential for managing the system timer hardware.

Introduction

The System Timer Control library provides functions to interact with the system timer hardware on the Raspberry Pi. The system timer is crucial for various timing-related tasks, including scheduling and precise timing measurements.

Hardware Addresses

- `MMIO_BASE` : Base address for Memory-Mapped I/O, set to `0xFE000000`.
- `SYSTIMER_BASE` : Base address for the System Timer, calculated as `MMIO_BASE + 0x003000`.

System Timer Registers

The library provides access to the following System Timer registers:

- `SYSTIME_CS` : System Timer Control/Status (0x00).
- `SYSTIME_CLO` : System Timer Counter Lower 32 bits (0x04).
- `SYSTIME_CHI` : System Timer Counter Higher 32 bits (0x08).
- `SYSTIME_C0` : System Timer Compare 0 (0x0C).
- `SYSTIME_C1` : System Timer Compare 1 (0x10).
- `SYSTIME_C2` : System Timer Compare 2 (0x14).
- `SYSTIME_C3` : System Timer Compare 3 (0x18).

Functions

```
uint32_t get_timerL0()
```

This function reads the lower 32 bits of the System Timer Counter.

```
uint32_t get_timerHI()
```

This function reads the higher 32 bits of the System Timer Counter.

```
uint64_t get_timerFULL()
```

This function returns the full 64-bit value of the System Timer Counter.

```
void set_timerC3(unsigned int value)
```

Set the System Timer Compare 3 register to a specified value. The timer interrupt will fire when the value matches the current timer value.

```
uint32_t get_timerC3()
```

This function retrieves the value from the System Timer Compare 3 register.

```
void resetCS(uint32_t value)
```

This function resets the specified bits in the System Timer Control/Status register. Use the `value` parameter to specify which bits to reset.

Generic Interrupt Controller (GIC) Documentation

This documentation provides an overview of the Generic Interrupt Controller (GIC) library for the Raspberry Pi. The GIC is a crucial component for managing interrupts on the system.

Introduction

The Generic Interrupt Controller (GIC) library provides functions to manage interrupts and the GIC hardware on the Raspberry Pi. The GIC plays a critical role in handling interrupts generated by various sources, including peripherals and devices connected to the Raspberry Pi.

Hardware Addresses

- `GIC_BASE` : Base address for the GIC, set to `0xff840000` .

GLCD - Distributor

- `GICD_BASE` : Base address for the GIC Distributor, calculated as `GIC_BASE + 0x1000`.
- `GICD_ISENABLERn` : Enable an interrupt. Offset = 0x100.

GICC - CPU Interface

- `GICC_BASE` : Base address for the GIC CPU Interface, calculated as `GIC_BASE + 0x2000`.
- `GICC_IAR` : GIC Interrupt Acknowledge Register. Offset = 0x0C.
- `GICC_EOIR` : GIC End of Interrupt Register. Offset = 0x10.

Functions

`void route_interrupt(uint32_t interrupt_id, uint8_t cpu_target)`

This function routes a specific interrupt to a CPU target. The `interrupt_id` is the ID of the interrupt, and `cpu_target` specifies the target CPU. The function sets the appropriate bits in the GIC Distributor.

`void enable_interrupt(uint32_t interrupt_id)`

Enables a specific interrupt using the GIC Distributor. The `interrupt_id` is the ID of the interrupt to enable.

`void setActiveInterrupt(uint32_t interrupt_id)`

Sets a specific interrupt as active. This function is used in managing interrupt states.

`uint32_t checkActiveInterrupt(uint32_t interrupt_id)`

Checks if a specific interrupt is active and returns `1` if it is active, `0` otherwise.

`void clearActiveInterrupt(uint32_t interrupt_id)`

Clears the active state of a specific interrupt.

`uint32_t readInterruptId()`

Reads the ID of the currently active interrupt from the GIC CPU Interface.

`void endInterrupt(uint16_t interrupt_id)`

Ends a specific interrupt. This function is used to signal the end of an interrupt with a given ID.

Updated Syscalls

1. **AwaitEvent(int eventType)** : This syscall allows a process to await a specific event. The process will return to the kernel and then call `KernelCreate`.

```
int AwaitEvent(int eventType) {  
    asm("svc 10"); // The Kernel needs to put the pid in x0  
    return;  
}
```

2. **GetRuntime()** : This syscall returns to the kernel and then calls `KernelCreate`. It can be used to retrieve runtime information.

```
int GetRuntime() {  
    asm("svc 11"); // The Kernel needs to put the pid in x0  
    return;  
}
```

3. **GetKernelRuntime()** : This syscall returns to the kernel and then calls `KernelCreate`. It's used to obtain kernel runtime information.

```
int GetKernelRuntime() {  
    asm("svc 12"); // The Kernel needs to put the pid in x0  
    return;  
}
```

Bug Fix

The provided code snippet appears to address a bug in the kernel code. You mentioned a fix for a kernel bug with the `send` operation on line 533 within the `syscall` function. It seems the bug was in this condition:

```
else if (PROCS[dest_p].queuesize >= QUEUESIZE){  
    // Handle the exception here and take appropriate actions.  
}
```

You would need to replace this snippet with the corrected code that fixes the bug. The exact fix depends on the nature of the bug and your specific requirements. Please provide the details of the bug and the desired correction so I can assist further.